P2P Collaboration App: Architecture & Protocol Spec (v0.1)

Shared contract for the team. **If code and this doc disagree, fix one of them the same day.** Anything marked **PROPOSED** is not frozen until the team confirms it.

Project: Network Application Development, Assignment 1: distributed, streaming-based client-server collaboration app over UDP. **Language:** Python (asyncio, explicit `socket` objects, `cryptography` for crypto). **Deadline:** the spec says "Sunday, November 17, 2026", but Nov 17 is a Tuesday. Confirm with the instructor; plan for **Sunday Nov 15** to be safe.

1. Architecture at a glance

A **hybrid P2P** system: a **tracker** (the server) handles login, presence, peer discovery, and global chat ordering. **Peers** talk directly to each other for DMs and file transfer. Everything runs on one shared **mini-QUIC style transport** built on raw UDP.

```
Browser UI (HTML + client-side JS)
     |  HTTP + Server-Sent Events on localhost (hand-written HTTP server, raw
TCP)
Client daemon  -------- encrypted UDP, tracker session ----------->  Tracker
     |  \
     |   `---------- encrypted UDP, peer sessions (direct) --------->  Other
client daemons
```

Layer stack (same library on client and tracker)

Layer	Responsibility	Module
0	Datagram socket wrapper + loss/delay/reorder simulator	`common/net.py`
1	Packet codec (header + payload)	`common/packet.py`
2	Connection: handshake, `conn_id`, keepalive, close	`transport/connection.py`
3	Reliable stream: seq, ACK/SACK, retransmit, reorder buffer	`transport/stream.py`
4	Multiple independent streams per connection (no head-of-line blocking)	`transport/connection.py`

Layer	Responsibility	Module
5	Congestion control (AIMD first)	`transport/congestion.py`
6	Encryption (AES-GCM)	`transport/crypto.py`
7	Application protocol (JSON messages)	`app/`
UI	HTTP/SSE gateway + web page	`client/gateway.py`, `web/`

The transport treats payloads as **opaque bytes**. It must never know what LOGIN or DM means.

Key decisions

Decision	Choice	Status
Topology	Hybrid: tracker (login, presence, global chat order, fallback relay) + direct P2P for DMs and files	Agreed
Transport	Custom reliable, message-oriented streams over UDP	Agreed
Build order	Bottom-up, starting with the peer-to-peer connection	Agreed
Concurrency	asyncio, with an explicit `socket.socket` handed to `create_datagram_endpoint(sock=...)`	Agreed
Control message encoding	JSON	PROPOSED
File data encoding	Raw bytes, no JSON/base64	PROPOSED
Reliability	Selective Repeat with SACK, per-stream	PROPOSED
Global chat ordering	Tracker assigns `gseq`	PROPOSED
Web UI	Local gateway: hand-written HTTP/1.1 on a raw TCP socket + SSE; no frameworks, no WebSockets	PROPOSED
Security	Level 1 (X25519 + AES-GCM) required; Level 2 (signed identity keys) only if time remains	PROPOSED

2. Conventions

Topic	Rule
Byte order	Big-endian (network order). `struct` formats start with `!` (no padding)
Text	UTF-8
Time	Unix epoch milliseconds (int)
Max UDP datagram	1200 bytes. Max payload = 1200 minus header size
Usernames	3 to 20 chars, `[a-z0-9_]`, unique and case-insensitive
Passwords	Server stores salted hash only (scrypt or PBKDF2)
Constants	All type codes, flags, timer values in `common/constants.py`. Never hard-code a number elsewhere
Code style	Type hints, docstrings on public functions, `black`-formatted

3. Transport protocol

3.1 Packet header (19 bytes) PROPOSED

`struct` format: `"!BBBIIHIH"`. One datagram = header + payload, sent with a single `sendto`.

Offset	Size	Field	Meaning
0	1	`version`	Protocol version, currently `1`. Reject others
1	1	`type`	Packet type (table below). `0` is invalid
2	1	`flags`	Bit set (below)
3	4	`conn_id`	The **receiver's** connection ID

Offset	Size	Field	Meaning
7	4	`pkt_num`	Per-connection, per-direction counter. **Increments on every packet sent, including retransmissions and ACKs.** Used for the encryption nonce and RTT sampling
11	2	`stream_id`	Which logical stream
13	4	`seq`	Per-stream sequence number (DATA only; `0` otherwise). Starts at **0**, counts packets
17	2	`length`	Payload size in bytes (must equal the actual remaining bytes)

Change from earlier sketches: `pkt_num` is new. Deriving the AES-GCM nonce from `seq` breaks for ACK/PING packets (they have no unique seq) and for retransmissions. A separate packet counter fixes both, at a cost of 4 bytes. **Confirm before freezing the header.**

3.2 Packet types

Code	Name	Sender	Payload	Reply
1	HELLO	Initiator	`sender_conn_id` (4B), `eph_pub` (32B), `nonce` (16B), `name_len` (1B), `name` (UTF-8)	HELLO_ACK
2	HELLO_ACK	Responder	`sender_conn_id` (4B), `eph_pub` (32B), `nonce` (16B), optional `sig` (64B, Level 2)	none
3	AUTH	Initiator	`sig` (64B), Level 2 only. Reserved	none
4	DATA	Either	One fragment of an app message	ACK
5	ACK	Either	`cum_ack` (4B), `window` (2B), `n_sack` (1B), then `n_sack` x (`start` 4B, `end` 4B), inclusive	none
6	PING	Either	`token` (8B)	PONG
7	PONG	Either	same `token`	none

Code	Name	Sender	Payload	Reply
8	FIN	Either	`reason` (1B)	none

This numbering **supersedes** earlier sketches (DATA was 3 before AUTH was added).

- HELLO carries `conn_id = 0` in the header, because the initiator does not know the responder's ID yet. After the handshake, every packet's header `conn_id` is the receiver's chosen ID.
- At security Level 0 (no crypto), `eph_pub` and `nonce` are zero-filled. The layout does not change.

3.3 Flags

Bit	Name	Meaning
0	EOM	Last fragment of an application message
1	ENC	Payload is encrypted (Level 1+)
2 to 7	reserved	Must be 0

3.4 Decode validation (drop silently, never crash)

Reject a datagram if: shorter than 19 bytes, wrong `version`, unknown `type`, reserved flag bits set, `length` does not match the remaining bytes, or payload exceeds the maximum. Log at debug level only.

3.5 Streams

Rule	Value
`stream_id` 0	Control (tracker connection) / DM + file control (peer connection)
`stream_id` 1	Global chat (tracker connection only)
New streams	Start at 2. **Initiator uses even IDs (2, 4, ...), responder uses odd IDs (3, 5, ...)** to avoid collisions

Rule	Value
Ordering	Strict in-order delivery **within** a stream only. No ordering across streams

Each stream has its own sequence space, send buffer, reorder buffer, and retransmit timer, which is how head-of-line blocking is eliminated.

3.6 Connection handshake

1. Initiator sends HELLO (header `conn_id=0`). Retry every 500 ms, up to 5 times.
2. Responder replies HELLO_ACK with its own `conn_id`. A **duplicate HELLO with the same `sender_conn_id` gets the same HELLO_ACK**, with no second session (idempotent).
3. Both sides derive keys (Level 1) and move to ESTABLISHED. Application DATA is accepted only in ESTABLISHED.

Session states: `NEW, HANDSHAKING, ESTABLISHED, CLOSING, CLOSED`.

3.7 Reliability (per stream)

Side	State
Sender	`next_seq`, `unacked` (seq to packet, send time, retries), send queue, RTT estimate, RTO timer
Receiver	`next_expected`, `reorder_buffer`, fragment buffer, receive queue

Receiver: if `seq < next_expected` or already buffered, drop it but **still ACK**. If beyond the window, drop. Otherwise buffer, then deliver in order and ACK.

ACK: `cum_ack = next_expected` (everything below has arrived), plus SACK ranges above the gap. Max 8 ranges per ACK.

Sender: on ACK, remove acked seqs. On timeout, retransmit the oldest unacked and double the RTO. Retransmit **only missing** packets. Fast retransmit when SACK shows 3 or more packets beyond a hole.

RTT/RTO: `SRTT` / `RTTVAR` smoothing as in TCP. **Karn's rule** applies, so never sample RTT from retransmitted packets (with `pkt_num`, a retransmission gets a new `pkt_num`, so this is easy to enforce).

Windows: `min(receiver window, cwnd)`. Start with a fixed window of 32 packets; add `cwnd` (AIMD) in Layer 5.

3.4b Default timers (all in `common/constants.py`)

Timer	Default
HELLO retry	500 ms x 5
Initial RTO / min / max	300 ms / 50 ms / 3 s
Max retransmissions before declaring peer dead	8
PING interval	5 s
Peer considered dead after silence	15 s
Unauthenticated tracker session timeout	30 s
Idle peer session timeout	120 s
Window	32 packets

3.8 Encryption plan PROPOSED

Level	What	Notes
0	Plaintext	For development and debugging
1	X25519 ephemeral key exchange, HKDF-SHA256, **AES-256-GCM** per packet	Header is authenticated data (AAD); payload encrypted. Nonce built from a role byte (initiator/responder) plus `pkt_num`. Never reuse a nonce under one key. Replay protection via a `pkt_num` window
2	Signed handshake with long-term Ed25519 identity keys, public keys distributed by the tracker	Stops impersonation and MITM between peers. Stretch goal

Known limit (document in the report): at Level 2 the tracker is a trusted introducer. The client pins the tracker's key (or trusts it on first use).

4. Application protocol (JSON inside DATA)

Every message is a UTF-8 JSON object: `t` (type), `id` (message ID), optional `reply_to`. Messages can span several packets; EOM marks the end, and the stream layer delivers the **complete** message.

4.1 Client and tracker

Message	Direction	Fields	Notes
LOGIN	C to S	`username`, `password`, `identity_pub` (Level 2)	Only accepted in CONNECTED state
LOGIN_OK	S to C	`peers[]`, `history[]`, `your_addr`	Sent **before** PEER_JOINED is broadcast
LOGIN_ERR	S to C	`reason`	Policy: max 3 attempts, then close
PEER_JOINED	S to all others	`username`, `addr`, `identity_pub`	
PEER_LEFT	S to all	`username`	Logout or timeout
CHAT_SEND	C to S	`text`	
CHAT	S to all (incl. sender)	`gseq`, `from`, `text`, `ts`	`gseq` gives the global order
CONNECT_REQ	C to S	`target`	Ask for an introduction
INTRO	S to both	peer info	Enables simultaneous open
RELAY	C, S, C	`to`, `payload`	Fallback when a direct path fails
LOGOUT	C to S		Followed by FIN

Message	Direction	Fields	Notes
ERROR	S to C	`code`, `text`	Codes: `BAD_CREDENTIALS`, `NAME_TAKEN`, `NOT_AUTHENTICATED`, `UNKNOWN_PEER`, `BAD_MESSAGE`

`peers[]` entries: `{username, addr:[ip,port], identity_pub}`. The tracker keeps the last N (say 50) chat messages for late joiners.

4.2 Peer and peer

Message	Fields	Reply
DM	`text`, `ts`	none (transport ACK means arrived)
FILE_OFFER	`transfer_id`, `name`, `size`, `sha256`, `stream_id`	FILE_ACCEPT / FILE_REJECT
FILE_ACCEPT	`transfer_id`	
FILE_REJECT	`transfer_id`, `reason`	
(chunks)	Raw bytes on the file's own stream. Order gives the offset	
FILE_END	`transfer_id`, `total_bytes`, `sha256`	FILE_DONE
FILE_DONE	`transfer_id`, `ok`, `reason`	
FILE_ABORT	`transfer_id`, `reason`	

File rules: the receiver verifies the hash only after receiving `total_bytes` (FILE_END can overtake the last chunks, since they are on different streams). Write to a temp file, then rename after the hash check. Progress = bytes received / `size`.

4.3 Tracker per-client state machine

State	Accepts	Drops
NEW	HELLO	everything else

State	Accepts	Drops
HANDSHAKING	HELLO retries, first DATA	
CONNECTED	LOGIN only	everything else
AUTHENTICATED	all app messages	
CLOSED	nothing	everything

5. Key flows

Startup and login

```
Client                                    Tracker
  |-- HELLO ---------------------------->|
  |<--------------------------- HELLO_ACK|
  |-- DATA{LOGIN} ---------------------->|
  |<--------------------------------ACK |
  |<-- DATA{LOGIN_OK, peers[], history} -|
  |-- ACK ------------------------------|    tracker then sends PEER_JOINED
to others
  |<== PING / PONG every 5 s ==========>|
  |-- DATA{LOGOUT} + FIN --------------->|    or timeout, then PEER_LEFT to
others
```

First DM to a peer: look up the peer in the local `peers[]`, send HELLO directly to its address, complete the handshake (Level 2: verify the signature against the tracker-provided key), then send `DM` on stream 0. Sessions are opened lazily and closed after the idle timeout. If both sides initiate at once, the **lexicographically smaller username** acts as initiator. If the direct HELLO fails after retries, fall back to RELAY via the tracker. HELLO from an address not in the local peer list is dropped.

File transfer: FILE_OFFER (stream 0) then FILE_ACCEPT then raw chunks on a new stream then FILE_END then FILE_DONE.

6. Module contracts (the interfaces teammates code against)

6.1 Transport API (what app code sees)

Call	Meaning
`conn = await connect(addr)` / `conn = await accept()`	Establish a session
`stream = conn.open_stream()`	New independent stream (ID parity rule applies)
`await stream.send(msg: bytes)`	Reliable, in-order within the stream. Fragments automatically
`msg = await stream.recv()`	Returns one **complete** message. Never blocks other streams
`conn.on_close(callback)`	Fires on FIN, timeout, or retry exhaustion

6.2 Datagram seam (Layer 0)

`send(data: bytes, addr)` and a callback `on_datagram(data: bytes, addr)`. The loss simulator wraps this seam (`--loss 0.2`, `--delay-ms`, `--reorder`).

6.3 Daemon and browser API PROPOSED

Endpoint	Purpose
`GET /`	Static web page
`GET /events`	Server-Sent Events stream: `peers`, `chat`, `dm`, `file_progress`, `status`
`POST /api/login`	`{username, password}`
`POST /api/chat`	`{text}`
`POST /api/dm`	`{to, text}`
`POST /api/file`	Raw body + file name and recipient in headers (browsers can't hand over a filesystem path)
`GET /api/state`	Current peers, connection status

7. Repo layout

```
common/    constants.py  packet.py  net.py
transport/ connection.py  stream.py  congestion.py  crypto.py
app/       messages.py  tracker.py  peer.py  files.py
client/    daemon.py  gateway.py
web/       index.html  app.js  style.css
tests/     test_packet.py  test_stream.py  test_tracker.py  ...
docs/      this spec, diagrams, report drafts
```

8. Work split

Track	Scope	Depends on	Mocks until ready
A: Transport core (critical path)	`packet.py`, loss simulator, handshake, reliable stream, multi-stream, congestion	none	n/a

Track	Scope	Depends on	Mocks until ready
B: Tracker + app protocol	Tracker state machine, login/auth, peer list, heartbeat sweeper, global chat with `gseq`, history replay, RELAY	A's API	In-memory fake `Connection` / `Stream` matching 6.1
C: Client + UI	Client daemon, peer session manager, DM, file transfer, HTTP/SSE gateway, web page	A's API, B's messages	Same fake transport, a scripted fake tracker
D: Crypto, tests, report	Level 1 crypto, integration tests, README, report, diagrams	A	Can start crypto in isolation

For 2 people: one takes A (then crypto), the other takes B + C. **For 3:** A / B + D / C.

To keep tracks independent, **write the fake transport first** (about 30 lines, in-memory queues). It lets B and C make progress while A builds the real thing, and swapping it in is one line.

Rubric mapping

Step	Where
1 Protocol design	This doc
2 Handshake	Track A
3 Auth + state	Track B
4 Concurrency	asyncio, Tracks A and B
5 Routing/streaming	Track B (global), Track C (DM)
6 Reliable file transfer	Tracks A and C
7 Resilience	Tracks A, B, C (timeouts, FIN, cleanup)
8 Security	Track D
9 HTTP UI	Track C

Step	Where
10 Tests + quality	Everyone, led by Track D

Schedule (as of Sun Nov 8; target Sun Nov 15)

Dates	Milestone
Nov 8 to 9	Freeze header + contracts. Codec + tests, loss simulator, fake transport, skeletons
Nov 9 to 11	Handshake and reliable stream (A). Tracker login/peer list/global chat on the fake transport (B). Client daemon shell (C)
Nov 11 to 12	Swap in the real transport. Multi-stream. DM end to end
Nov 12 to 13	File transfer, Level 1 encryption
Nov 13 to 14	Web UI, integration tests
Nov 14 to 15	README, report, buffer

Transport acceptance tests (must pass before integration): 1 MB file with 20% loss + reordering arrives with an identical hash; a chat message sent mid-transfer arrives promptly (no head-of-line blocking); `kill -9` on one peer leads to a clean timeout on the other; duplicated and replayed packets are ignored.

9. Open decisions

1. Real deadline: Sunday Nov 15 or Tuesday Nov 17?
2. Add `pkt_num` to the header (recommended)?
3. LOGIN_ERR policy: 3 attempts then close, or close immediately?
4. Tracker key: pinned or trust-on-first-use?
5. Aim for Level 2 authentication, or stop at Level 1?
6. How many teammates, and who owns each track?